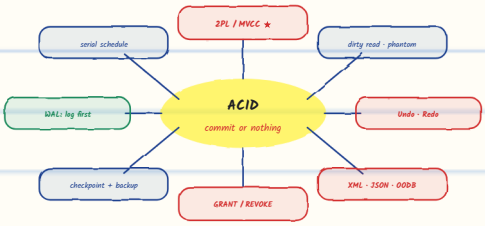
Date: 13.10.2026
topper's notes

Transactions, Concurrency, Recovery & Security

ACID · 2PL · MVCC · isolation levels · WAL · GRANT · XML/JSON · OODB/ORDB

Eleven sheets: the transaction and its four properties, the problems concurrency creates, four ways of stopping them, the four isolation levels, how the log rebuilds a crashed database, who may see what — and the semi-structured and object worlds after that.



↑ the whole topic: red = high-yield, green = the recovery engine

→ Syllabus coverage

The syllabus names this lesson clause by clause: "Transaction processing; DBMS concurrency control; database recovery; database security and access control; XML, XPath and XQuery; object-oriented and object-relational databases." **Every clause is taken in full here.** Earlier lessons designed the relations, gave the SQL that reads them and put their bytes on disk; this one must make all of that survive many users, a failure and an intruder at once — can ten students log in, one crash mid-update and one try to read a colleague's salary, and the database still be correct?

High-yield list

① ACID = Atomicity, Consistency, Isolation, Durability
② All or nothing: COMMIT or ROLLBACK
③ Active → Partially committed → Committed
④ Anomaly loop ends at Failed → Aborted
⑤ Serialisable = as if one at a time
⑥ Dirty · lost update · non-repeatable · phantom
⑦ S+S yes · S+X no · X+X never
⑧ 2PL: grow then shrink, still deadlocks
⑨ RU < RC < RR < Serializable = safer, slower
⑩ WAL: log record first, data page second
⑪ UNDO uncommitted, REDO committed
⑫ Checkpoint + full / incremental backup

Legend yellow = key term · red circle = must-memorise · **purple** = memory trick · **red** = exam trap · **green** = worked example

→ Contents

✓ Transactions & the ACID properties … p. 2
✓ Schedules & concurrency problems … p. 3
✓ Locks: S/X, 2PL & deadlock … p. 4
✓ Timestamp, optimistic, MVCC … p. 5
✓ Isolation levels, standard vs real … p. 6
✓ Recovery: log, WAL, checkpoint … p. 7
✓ Access control & security … p. 8
✓ XML, JSON, OODB/ORDB … p. 9
✓ Night-before sheet … p. 10
✓ Practice MCQs + written … p. 11

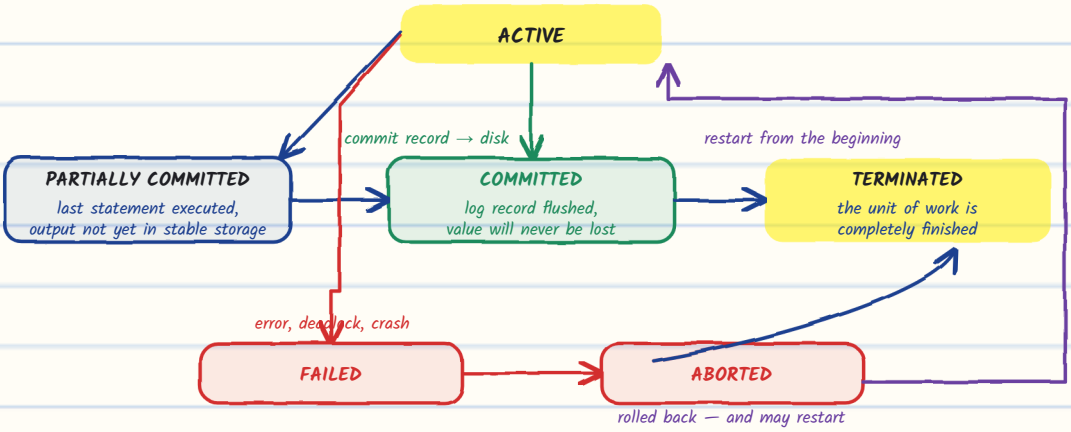
1 Transactions: One Logical Unit of Work, and Its Four Properties

all or nothing ★

Definition A **transaction** is a sequence of operations treated as a single logical unit of work, delimited by BEGIN / COMMIT on one side and COMMIT / ROLLBACK on the other: either **every** operation takes effect or **none** does. Transferring Tk 1000 is two writes — take from account A, add to account B — and if only the first ever reaches disk the money is destroyed, which is why the two writes must be **one** transaction.

```
BEGIN TRANSACTION;
  UPDATE account SET balance = balance - 1000 WHERE accno = 'A-101';   -- 1
UPDATE account SET balance = balance + 1000 WHERE accno = 'B-207';   -- 2
COMMIT;   -- both or neither
```

Property	What it promises	Who enforces it
Atomicity	All or nothing. A failure anywhere rolls the whole unit back; a partly applied transfer is never visible.	recovery manager (UNDO)
Consistency	All declared integrity constraints hold before and after. **They are written by the user** — the DBMS invents none.	constraints, the user's SQL
Isolation	Each transaction appears to run alone. This is the property concurrency control exists to deliver.	concurrency control
Durability	Once committed, a change survives any crash — power loss, reboot, media failure.	recovery manager (REDO, WAL)



the full transaction-state diagram — five states, and the loop back from ABORTED

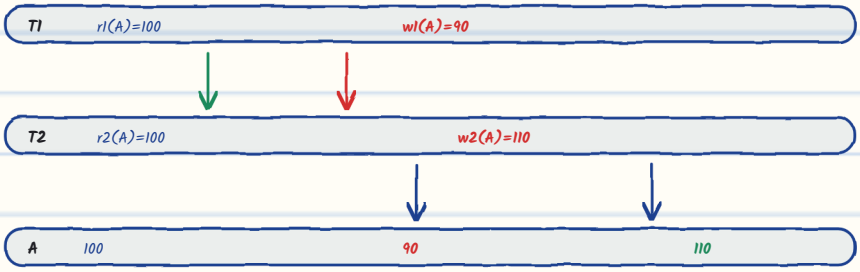
Exam trap ✍ five things drawn wrong **1. FAILED** is reached from **ACTIVE** — not from *Partially committed*; the partial state exists only so a failure detected at output can still undo. **2. ABORTED** follows **FAILED**, never the reverse. **3. TERMINATED** is the real end state, reached from **COMMITTED** and from **ABORTED** — "Committed is the last state" is wrong. **4.** An **ABORTED** transaction may be restarted from the beginning. **5. Commit** means the log record is on stable storage — not that every dirty page is flushed.

Memory trick **"Act like a committee: Atomic never splits, Consistent obeys the rules, Isolated keeps its own counsel, Durable survives the walk out."** And for the diagram: **"Active starts, Partial is the doorway, Committed is the promise, Terminated is the door closing; only failure opens the side exit to Aborted."**

2 Schedules, Serialisability, and the Five Anomalies

as if one at a time ★

Definitions A **schedule** is the time order in which the operations of several transactions interleave. A **serial schedule** runs one transaction to completion, then the next — always correct, useless in practice. A **concurrent** schedule mixes them for **throughput** and is **serialisable** if it is equivalent to some serial schedule. That one word is the whole objective.



two defects in one schedule: a dirty read at r2(A), then a lost update at w2(A)

Anomaly	Exactly what goes wrong	Fixed by
Dirty read	T2 reads an item T1 wrote but did not commit; T1 then rolls back, so T2's answer rests on a value that never existed.	S→X conflict · RR+
Lost update	Both read A=100 then both write; the second write overwrites the first unseen. The wrong value survives even though both committed.	X held to commit · delayed write
Non-repeatable read	The same read query returns different values twice in one transaction because a row was updated in between.	S held to commit · RR+
Phantom read	A new or deleted row shows up in a repeated range query. No value changed — the row's existence did.	range locks · Serializable
Inconsistent (incorrect summary) analysis	An aggregate reads part of the table before an update and the rest after it, so the total never existed at any instant.	S lock on every row read

Exam trap ✍ four **1. Lost update is the dangerous one, because both transactions commit** — no rollback can undo it. **2. Lost update is not removed by the standard isolation-level table**, which lists read anomalies only. **3. Phantom is about rows arriving, not values changing** — a value change is non-repeatable read. **4.** "Concurrency control is only about dirty reads" is far too narrow; an MCQ listing **all four plus inconsistent analysis** wants the standard set of five.

Memory trick "**Dirty = too early, Lost = too late, Non-repeatable = the value changed, Phantom = the row arrived.**" Two are about reading, two about writing; "phantom happens when the value changes" describes nothing at all.

The other half of the answer **Serialisability is a property of the schedule, not of the program.** So the DBMS does not stop interleaving; it accepts any interleaving whose effect is indistinguishable from a serial one.

3 Lock-Based Control: the Compatibility Matrix, 2PL, and Deadlock

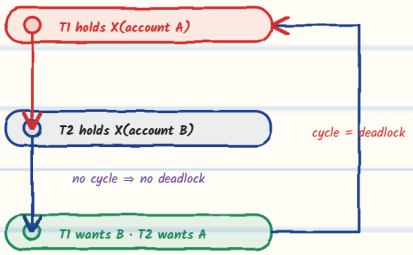
the most tested table here ★

The two locks A **shared lock S** is taken to read an item — many transactions may hold it together. An **exclusive lock X** is taken to write it — only one may hold it. The item is labelled **X** because a lock is **upgraded by adding, never by replacing**.

Held \ Requested	S(A)	X(A)
S(A) held	compatible ✓	conflict ✗
X(A) held	conflict ✗	conflict ✗

Exam trap ✍ three conclusions everyone gets wrong **1.** Only **one** cell is compatible — **S with S**. "S with X is compatible for reading" is false: the X holder may be changing the value. **2.** Two transactions wanting **X** never meet, so **no read-write anomaly survives 2PL**. **3.** A transaction holding S and asking for X must **release S first**, which may already have closed the shrinking phase.

Two-Phase Locking (2PL) ✍ the definition Every transaction runs in **two phases**: the **growing phase**, which may **acquire** locks but release none, and the **shrinking phase**, which may **release** locks but acquire none. It commits at the end of the shrinking phase.



wait-for graph: a cycle is a deadlock, and 2PL does not rule one out

Exam trap ✍ what 2PL guarantees **Guarantees:** **conflict-serialisable** schedules (no cycle forms in the precedence graph) — but **not** strict or recoverable ones: a lock may go before commit; **strict 2PL** fixes that. **Does not guarantee:** freedom from deadlock or from starvation — two transactions locking in opposite order deadlock under 2PL, the commonest wrong claim here.

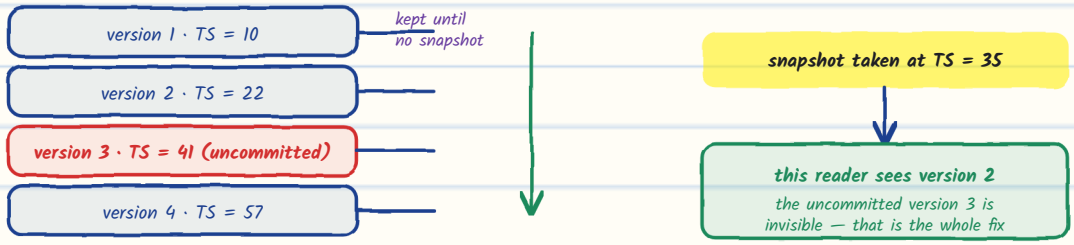
Memory trick **"Grow then shrink: first only open doors, then only close them." Strict 2PL** keeps every **X** lock to commit — **no cascading rollback**; **rigorous** keeps S and X. **Conservative 2PL** declares every needed lock **before the transaction starts**, so a missing lock means it never starts: **deadlock-free**, at a large cost in concurrency.

Dealing with deadlock	Method
Prevention (before it happens)	Impose a **fixed global order** on which item may be locked first, and a cycle becomes impossible; the data-engineering alternatives are **wait-die** and **wound-wait**, driven by **timestamps**.
Detection (after it happens)	Maintain a **wait-for graph**: an edge Ti→Tj means Ti waits for Tj. Search it for a **cycle**; on finding one, **abort one transaction** in it — usually the one with the least work — release its locks and let it restart.

4 Beyond 2PL: Timestamp Ordering, Optimistic Control, and MVCC

no locks, still serialisable ★

Technique	How it works	The catch
Timestamp Ordering (TO)	Every transaction is stamped TS(T) = (time, counter) so **TS is unique and comparable**. An operation is allowed only if it respects the item's last write; otherwise the transaction is aborted and retried.	Deadlock-free, but **work can be thrown away** and starvation is possible.
Optimistic (validation)	Assume conflicts are **rare**: run with no locks at all, record the items read and written, and at commit time **validate** that nothing read has changed.	Poor fit for **write-heavy** work — the answer to validation is often "abort".
MVCC (multi-version)	Keep several versions of each row stamped with their writer. A reader takes a **snapshot** and reads the version it should see, so **no reader blocks**; only two writers to one row conflict.	Old versions must be reclaimed, and **write-skew** must still be rejected at commit.



one row, four live versions — the reader's snapshot timestamp picks which one

MVCC: the row keeps several versions; the reader's snapshot timestamp decides which one it sees

Exam trap ✍ MVCC is not "no locking" ✍ three

1. "MVCC uses no locks at all" is false — readers take none, but two transactions **writing** the same row still conflict. **2.** Its selling point is stated wrongly very often: it removes **reader–writer** blocking, not write–write conflicts. **3.** "MVCC prevents every anomaly" is false — snapshot isolation still admits **write-skew**, so real engines detect it and abort with a serialisation failure the client retries.

Where each one is used **PostgreSQL** keeps old tuple versions and reclaims them with VACUUM; **Oracle** keeps undo in a rollback segment and uses it to reconstruct a read-consistent snapshot; **MySQL InnoDB** keeps undo logs and hides uncommitted versions; **SQL Server** keeps row versions in tempdb for READ_COMMITTED_SNAPSHOT. Locking and versioning are not rivals — every one of these systems uses both.

Memory trick **"Locks queue, timestamps judge, optimism hopes, versions cheat."** — 2PL makes transactions wait for each other, timestamp ordering makes a late transaction restart, optimistic control makes it hope nothing changed, and MVCC shows everyone an old copy so nobody waits at all.

5 Isolation Levels: SQL Standard, and What Real Engines Actually Do

one knob, four positions ★

Definition The **isolation level** is the degree to which one transaction's work is hidden from another — the consistency against throughput dial. Turn it right for the strongest guarantee and the slowest execution; turn it left for speed and accept the anomalies. Every level below the top one is defined precisely by **which anomalies it permits**.

Isolation level	Dirty read	Non-repeatable	Phantom	Speed
Read Uncommitted	allowed	allowed	allowed	fastest
Read Committed	no	allowed	allowed	fast
Repeatable Read	no	no	allowed	slower
Serializable	no	no	no	slowest

```
SET TRANSACTION ISOLATION LEVEL REPEATABLE READ;
BEGIN TRANSACTION;
  SELECT balance FROM account WHERE accno = 'A-101';
  -- a second identical SELECT here must return the same value
COMMIT;
```

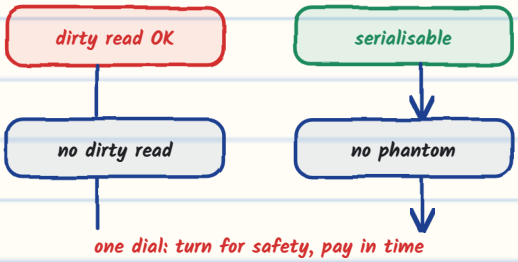
Exam trap ✍ the standard table is not what the product does ✍ four

1. The table above is the **SQL standard's** definition, where Repeatable Read still permits phantoms — it stops a row's value changing, not a row arriving. **2. MySQL InnoDB** goes further: next-key (gap + record) locking blocks phantoms at Repeatable Read. **3. PostgreSQL** implements Repeatable Read as snapshot isolation, which also blocks phantoms, and adds a true Serializable level with the write-skew check. **4. SQL Server** stays closest to the standard, and phantoms remain possible at Read Repeatable. "Repeatable Read forbids phantoms" is therefore **engine-dependent** — a favourite of ambiguous MCQs.

Lost update, the one the table cannot show

The three columns are about reading. A **lost update** is a write conflict, and the standard does not remove it by raising the level: in many engines the same level still loses an update unless the row is locked, or the application does a delayed (optimistic) write — read the value, then write with `WHERE balance = <the value you read>` and retry if it matches nothing.

Memory trick "**Read Uncommitted is the gossip, Read Committed is the news, Repeatable Read is the record of that day, Serializable is the court transcript.**" The ladder in one line: more isolation = more waiting. For a written answer the strongest sentence is: "raising the level replaces logical errors with physical waiting, so the choice is between correctness and throughput."



the ladder runs from a dirty read to a serial schedule

6 Recovery: the Log, the WAL Rule, UNDO/REDO, Checkpoints and Backups

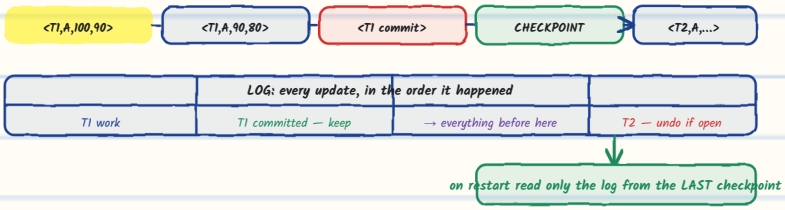
the crash sheet ★

Failure	What happened	Handled by
Transaction failure	Logic error, constraint violation, deadlock victim, or the user pressing abort.	**UNDO** in place — the DBMS does not restart
System crash	Power loss, OS kill, hardware fault. RAM is gone; **what was already on disk survived**.	**REDO** from the log
Media / disk failure	Bad sectors, head crash, a lost volume. The database itself is damaged or gone.	restore from **backup** + **REDO** the log

What a log record holds Each record carries a **transaction id**, a **previous-log-record address** (so the log is a linked list in both directions — back for rollback, forward for redo), the **data item**, its **old** and **new value** (before- and after-image), plus the `<T_i, X, V_old, V_new>` commit/abort marker. A **checkpoint** is a log record, not a copy of the database.

Exam trap ✍ the WAL rule ✍ three **Write-Ahead Logging rule: the log record for an update must reach stable storage before the updated data page is allowed to. 1.** "Log after writing to the database" inverts the rule and is the wrong option in any MCQ. **2.** WAL makes **durability** and **atomicity** cheap; it is **not** a backup. **3.** "A checkpoint copies the whole database" is a misconception — a **fuzzy checkpoint** only writes a record naming the dirty pages, so nothing is blocked.

Update policy	When the data page is written	After a crash we must...
Deferred update	Only **after commit**.	**REDO** only — nothing uncommitted is on disk.
Immediate update	As soon as the buffer is flushed, **before commit**.	**UNDO** uncommitted, **REDO** committed.



one log, one checkpoint: recovery replays only what follows the last checkpoint

Backups and the rest ✍ **Full** — copy everything; slow, restores fast. **Differential** — what changed **since the last full**; grows with time. **Incremental** — what changed **since the last backup of any kind**; smallest, but a restore needs the whole chain in order. **Shadow paging** keeps a shadow copy of every page and switches to it atomically at commit — no logging needed for rollback — but the old page chains become garbage. **ARIES**, what real systems use, runs three passes: **analysis** from the last checkpoint, **redo** (repeat history), **undo** (roll back the losers).

Memory trick "**Log first, data second**" is the WAL rule in three words and the whole reason a log exists. "**Undo the uncommitted, redo the committed**" covers every crash case: a transaction that neither committed nor aborted before the crash is treated as aborted, which is why **there is never any doubt after a power failure**.

7 Access Control and Security: Who May See What

DAC · MAC · RBAC ★

The two halves **Authentication** answers "who are you?" — password, biometric, one-time code, a second factor. **Authorisation** answers "what may you do?" — read, insert, update, delete, or just some columns of some rows. A database is **secure** only when both halves are present: a perfect authorisation scheme behind an open door is worthless.

Model	Who decides access	Its point, and its weakness
DAC — Discretionary	The **owner** of each object. Each user may grant their own rights onward.	Flexible and cheap, but a user can give away the very right they hold — the classic trojan-horse route.
MAC — Mandatory	**The system**, from a security label: classification × compartment (Top Secret, Secret, Confidential, Unclassified).	Nobody may **reduce** a classification, not even the owner — military and government data.
RBAC — Role-Based	**Roles**: permissions attach to a job function, and users are assigned to roles.	Kills the "who may do this?" explosion — change the job, change the role, touch no privilege list.

```
CREATE ROLE Auditor;
GRANT SELECT ON Employees TO Auditor;
GRANT UPDATE (salary) ON Employees TO Auditor;
REVOKE SELECT ON Payslips FROM intern;
```

Exam trap ✍ five **1. GRANT** gives a privilege, **REVOKE** takes it away — not interchangeable, and REVOKE names the grantee. **2.** In **MAC** rights are decided by the **system**, so a user cannot grant anything; in **DAC** rights travel with ownership. **3. RBAC is about roles, not rows** — row-level access is a different feature. **4.** A **view** is a cheap second line of defence: expose only the columns and rows the user needs and grant on the view, not the table. **5. Encryption** is confidentiality, **auditing** is accountability, **integrity constraints** are correctness — three different jobs, and the exam likes them confused.

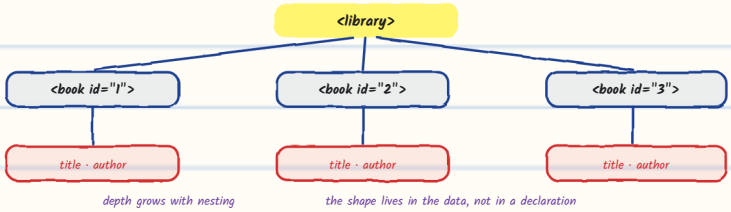
Threats and their answers **At rest** — disk or backup theft: **encryption**. **In transit** — a network tap: **TLS**. **A careless user**: views, roles, least privilege. **A careless developer**: SQL injection — so queries are **parameterised**, never built by concatenating user input. **An insider**: an **audit trail**, so accountability exists after the fact. **A careless admin**: the DBA is all-powerful by design, so duties are separated — recovery administration is often split from ordinary DDL.

Least privilege, in practice ✍ Grant the **minimum** privilege that lets the job be done, and no more: read-only for a viewer, one table for an operator, `GRANT UPDATE(column)` rather than the whole table when only one field changes, and never `SELECT *` rights on a payroll table. A privilege once granted is inherited by every backup, every replica and every audit log the moment you make it.

Memory trick "**DAC is the owner, MAC is the label, RBAC is the job-title.**" Say it in that order and all three are fixed. And "**least privilege is the whole policy**" — every grant you do not need is a hole you did not have to fix.

8 Semi-Structured Data: XML, XPath, XQuery, JSON — and OODB vs ORDB

the last two clauses ★



XML is self-describing: the shape is part of the data

The two query languages — do not confuse

XPath is a small path-and-predicate language over one document: `/library/book/title`, `/library/book[@id="2"]/author`, `//author[text()="CLRS"]`. **XQuery** is a full query language with FLWOR — `for, where, order by, return` — over one or many documents, and it can compute and join. "XPath joins two documents" is simply wrong.

```
for $b in /library/book
where $b/author = "CLRS"
order by $b/title
return $b/title/text()
```

Exam trap ✍ three **1.** In XPath an **attribute is text**: `[@id="2"]` matches the string "2", so `[@id=2]` is the wrong spelling. **2.** `//` means "anywhere below", not "the second level". **3.** JSON has no comments and no schema by default, and JSONPath is far thinner than XPath: it navigates and filters but has no FLWOR and no joins.

Relational, XML, JSON side by side

Relational: structured, schema declared first, rows of values, joins and set operations for free, but a shape change means a schema change. **XML**: semi-structured, self-describing, nesting and heterogeneous records, validated by a **DTD** or an **XSD**. **JSON**: semi-structured too, lighter than XML, native to the web, used by nearly every API. The DBMS answer to all three is the same: store the document whole, or shred it into relations — which is what "XML to relational mapping" means.

```
{"library":{"books":[
  {"id":1,"title":"Database Systems","author":"Navathe"},
  {"id":2,"title":"Intro to Algorithms","author":"CLRS"}]}}
$.library.books[*].title
```

Point	OODB	ORDB
Data model	**pure objects** with identity	tables + types
Language	OQL / ODL	SQL + extensions
Examples	ObjectStore, db4o, ObjectDB	PostgreSQL, Oracle, DB2

Exam trap ✍ the last three **1.** An **OODB keeps object identity** — two objects equal in value are still different objects — exactly what a relational key throws away. **2.** An **ORDB is not "a relational database with a picture attached"**: it adds **user-defined types**, **nested tables**, **object identity** and **inheritance** while keeping the relational core. **3.** "Object databases are faster" only means anything for **deep, complex object graphs**; for ordinary business rows with joins, a mature relational or object-relational engine still wins on tooling and cost.

Night-Before-the-Exam Sheet

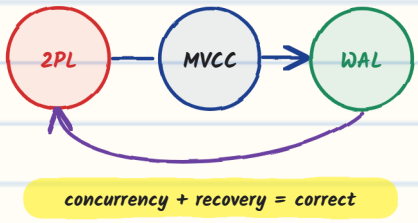
Fact sheet

ACID = Atomicity … … Consistency, Isolation, Durability
Commit = all, Rollback = none Active → Partially committed
→ Committed → Terminated Active → Failed → Aborted
Serial = one at a time Serialisable = as if serial
Dirty: too early Lost update: too late
Non-repeatable: value changed Phantom: row arrived
S + S = compatible S + X, X + X = conflict
2PL: grow, then shrink 2PL ≠ deadlock-free
Strict / rigorous 2PL ⇒ strict Conservative ⇒ no deadlock
Wait-for cycle = deadlock Abort one; or fix order
Timestamps: may abort OCC: validate at commit
MVCC: snapshot readers RU < RC < RR < Serial
RR standard: phantom OK InnoDB / PG: no phantom
WAL: log first, data 2nd UNDO open, REDO committed
Deferred ⇒ REDO only Immediate ⇒ UNDO + REDO
Checkpoint ⇒ replay past it Full / differential / incremental
DAC owner · MAC label RBAC job-title
GRANT gives, REVOKE takes Least privilege is the policy
XPath = paths · XQuery = FLWOR OODB: objects + OQL
ORDB: tables + UDT Constraints give consistency

→ Top 12 one-liners for MCQs

1. A transaction is all-or-nothing
2. **Committed is not the last state** — Terminated is
3. **Failed from Active**; Aborted from Failed
4. Serialisable = as some serial schedule
5. Only **S with S** is compatible
6. 2PL is serialisable, **not deadlock-free**
7. RR stops a value changing, **not a row arriving**
8. MVCC stops reader–writer blocking only
9. **WAL: log first**
10. Deferred ⇒ **REDO only**; immediate ⇒ both
11. **DAC owner, MAC label, RBAC job-title**
12. Only **XQuery** has FLWOR

Top traps claiming **2PL prevents deadlock** · drawing **Failed after Partially committed** · calling **phantom** a changed value · saying **Repeatable Read forbids phantoms** with no engine named · **logging after** the data write · **UNDO for committed** work · **checkpoint = full copy** · **MAC with DAC** · **FLWOR on XPath**.



three tools, one promise

All tricks together Act like a committee · Active starts, Terminated ends · Dirty too early, Lost too late · Grow then shrink · Locks queue, timestamps judge, optimism hopes, versions cheat · More isolation = more waiting · Log first, data second · Undo the uncommitted, redo the committed · DAC the owner, MAC the label, RBAC the job-title.

✓ Practice MCQs (exam pattern, 10 marks)

try first, then check

1. A transaction reaches **Committed** state when:
   (a) Its last statement has executed (b) All updates are flushed to the pages
   (c) Its commit log record is on stable storage (d) All locks it holds are released

2. Which lock pair is **compatible**?
   (a) S(A) with X(A) (b) X(A) with S(A)
   (c) S(A) with S(A) (d) X(A) with X(A)

3. Two-Phase Locking guarantees:
   (a) Freedom from deadlock (b) Conflict-serialisable schedules
   (c) Freedom from cascading rollback only (d) That no transaction is ever aborted

4. A **phantom** occurs when:
   (a) A row's value is updated twice (b) An uncommitted value is read
   (c) Two updates overwrite each other (d) New rows appear in a repeated range query

5. Under the SQL standard, **Repeatable Read** still permits:
   (a) Dirty reads (b) Lost updates
   (c) Phantom reads (d) Nothing — it is fully serialisable

6. The Write-Ahead Logging rule requires:
   (a) The data page is written before the log record (b) The log record reaches stable storage before the updated page
   (c) Both are written in the same disk block (d) A checkpoint after every commit

7. With **immediate update**, a crash requires recovery to:
   (a) REDO only (b) UNDO only
   (c) UNDO the uncommitted and REDO the committed (d) Restore the last full backup only

8. **Write-skew** is possible under:
   (a) Serialisable (b) Snapshot isolation
   (c) Two-phase locking (d) Conservative 2PL

9. Which statement about **MAC** is correct?
   (a) The owner may grant rights to anyone (b) Rights are fixed by a system security label
   (c) Rights are attached to job roles (d) A user may always lower a classification

10. Which language has **for / where / return**?
    (a) XPath (b) XQuery
    (c) SQL (d) JSONPath

Answer key → 1-c · 2-c · 3-b · 4-d · 5-c · 6-b · 7-c · 8-b · 9-b · 10-b

Written question Define a transaction and state the four ACID properties with **who enforces each**. Give the **lock compatibility matrix** and explain **two-phase locking**, then say what 2PL does **not** guarantee and how a deadlock is detected. State the **WAL rule** and the difference between **deferred** and **immediate** update on crash. 10 marks

1. **Transaction and ACID (3)** — a **sequence of operations treated as one logical unit of work**, ended by COMMIT or ROLLBACK. **Atomicity** all or nothing, enforced by the recovery manager's UNDO. **Consistency** all declared integrity constraints hold before and after — the constraints are the user's, not the DBMS's. **Isolation** each transaction appears to run alone, enforced by concurrency control. **Durability** a committed change survives a crash, enforced by the recovery manager through the log and the WAL rule. 3
2. **Lock matrix (2)** — S(A) with S(A) is the **only** compatible pair; S with X, X with S and X with X all conflict. Many readers, one writer, never two writers. 2
3. **2PL, and its limits (2)** — **growing phase** acquires and releases nothing, **shrinking phase** releases and acquires nothing; the schedule is **conflict-serialisable**, but basic 2PL may release a lock before commit, so it is not necessarily recoverable; **strict 2PL** (X locks held to commit) prevents cascading rollback. It does **not** guarantee **freedom from deadlock** or from starvation. 2
4. **Deadlock (1)** — build a **wait-for graph** with an edge $T_i \to T_j$ when T_i waits for T_j; a **cycle** means deadlock, detected by a periodic search, broken by **aborting one transaction** and restarting it. Prevention uses a fixed lock order or the wait-die / wound-wait timestamp schemes. 1
5. **WAL and the two update policies (2)** — the log record of an update must be on stable storage before the updated data page is. Under **deferred update** no page is written before commit, so recovery does **REDO only**; under **immediate update** pages are flushed early, so recovery does **UNDO the uncommitted and REDO the committed**. A checkpoint marks the point from which the log must be replayed. 2

→ Before the exam ✓

- ✓ Can draw the five-state transaction diagram
- ✓ Can name all five concurrency anomalies
- ✓ Can reproduce the lock compatibility matrix
- ✓ Can state what 2PL does not guarantee
- ✓ Can state the WAL rule word for word
- ✓ Can separate XPath from XQuery, and OODB from ORDB